Note sur le jeu de données Spark

Cours d'optimisation Spark avec Scala

Objet de la note. Ce document décrit les données créées par le script generate_course_data.scala. Le jeu de données simule une activité de commerce en ligne et contient volontairement plusieurs situations utiles pour apprendre à diagnostiquer et optimiser un programme Spark.

Vue d'ensemble

Dossier	Contenu	Volume par défaut	But pédagogique
countries	Référentiel des pays	10 lignes	Broadcast join et dimension de référence
customers	Clients	200 000 lignes	Jointures, filtres et projection
orders_partitioned	Commandes partitionnées par date	2 000 000 lignes	Partition pruning, pushdown, skew et column pruning
orders_small_files	Copie fragmentée des commandes	2 000 000 lignes	Problème des petits fichiers
events	Événements utilisateurs	3 000 000 lignes	Agrégations, cache, partitionnement et skew

Relations principales

- customers.country_code → countries.country_code
- orders.customer_id → customers.customer_id
- events.user_id → customers.customer_id
- orders.country → countries.country_code

Paramètres de génération

Variable	Valeur par défaut	Rôle
COURSE_DATA_PATH	./spark-course-data	Répertoire racine de sortie
CUSTOMER_COUNT	200000	Nombre de clients
ORDER_COUNT	2000000	Nombre de commandes
EVENT_COUNT	3000000	Nombre d'événements
NORMAL_PARTITIONS	32	Nombre normal de partitions d'écriture
SMALL_FILE_PARTITIONS	240	Nombre élevé de partitions pour le laboratoire small files
OVERWRITE	true	Écrase les sorties existantes si true

Table countries

Petit référentiel statique de dix pays. Il constitue le candidat naturel pour une jointure par broadcast.

Colonne	Type	Description et règle de génération
country_code	String	Code à deux lettres parmi TG, FR, IT, DE, BE, GH, CI, SN, US et GB.
country_name	String	Nom complet du pays.
region	String	Europe, West Africa ou North America.
active	Boolean	Tous les pays sont actifs sauf le Royaume-Uni.

Table customers

Dimension client générée avec spark.range. Les valeurs sont déterministes et réparties régulièrement afin de faciliter les comparaisons entre exécutions.

Colonne	Type	Description et règle de génération
customer_id	Long	Identifiant compris entre 1 et CUSTOMER_COUNT.
customer_name	String	Valeur Customer suivie de l'identifiant.
country_code	String	Répartition cyclique entre les dix codes pays.
segment	String	VIP si l'identifiant est multiple de 20, Business s'il est multiple de 4, sinon Standard.
active	Boolean	False pour un identifiant multiple de 17, true dans les autres cas.
signup_date	Date	Date comprise dans une fenêtre de 1 460 jours à partir du 1er janvier 2022.
email	String	Adresse synthétique customer<ID>@example.org.

Exercices associés

- Joindre customers à countries et comparer un BroadcastHashJoin à un SortMergeJoin.
- Filtrer les clients actifs ou un segment particulier avant une jointure.
- Sélectionner uniquement les colonnes utiles pour observer le column pruning.

Tables de commandes

Les deux dossiers de commandes contiennent les mêmes colonnes et le même volume logique. Leur organisation physique diffère pour permettre deux familles d'exercices.

Colonne	Type	Description et règle de génération
order_id	Long	Identifiant égal à l'index de génération augmenté de 1 000 000.
customer_id	Long	Clé client. Environ 22 % des lignes utilisent volontairement la valeur 42.
status	String	PAID, PENDING ou CANCELLED avec une distribution pseudo-aléatoire reproductible.
amount	Double	Montant aléatoire arrondi entre 10 et 500.
product_category	String	Books, Office, Electronics, Food ou Home.
country	String	Code pays calculé à partir d'un hash de l'identifiant.
order_date	Date	Date comprise dans les 365 jours suivant le 1er janvier 2026.
debug_payload	String	Colonne volontairement large et inutile, composée d'un texte et d'un hash SHA-256.

orders_partitioned

Les commandes sont repartitionnées sur order_date puis écrites avec partitionBy(order_date). Chaque date devient donc une partition de répertoire.

- Un filtre sur order_date permet le partition pruning.
- Un filtre sur status peut être poussé vers le lecteur Parquet.
- L'absence de debug_payload dans un select permet de constater le column pruning.

orders_small_files

La même table est écrite avec 240 partitions par défaut et sans partitionnement métier. Cette fragmentation produit volontairement beaucoup de fichiers relativement petits.

- Comparer le temps de listing et de lecture avec orders_partitioned.
- Tester coalesce ou repartition avant l'écriture.
- Comparer le nombre et la taille des fichiers obtenus.

Table events

Table de faits représentant l'activité des utilisateurs pendant octobre 2026. Elle est partitionnée physiquement par event_date.

Colonne	Type	Description et règle de génération
event_id	Long	Identifiant issu de spark.range.
user_id	Long	Référence customer_id. Environ 18 % des lignes utilisent volontairement la valeur 42.
event_type	String	view, search, cart, purchase ou support.
value	Double	Valeur pseudo-aléatoire arrondie entre 0 et 100.
event_date	Date	Date comprise entre le 1er et le 31 octobre 2026.
event_time	Timestamp	Horodatage réparti sur les 31 jours d'octobre 2026.

Anomalies pédagogiques volontaires

Anomalie	Emplacement	Effet attendu	Optimisations à tester
Clé chaude 42	orders et events	Partition très volumineuse pendant une jointure ou une agrégation	AQE skew join, broadcast lorsque possible, selective salting
Colonne large inutile	orders.debug_payload	Davantage d'entrées-sorties et de mémoire si elle est lue	Projection anticipée et column pruning
Nombre élevé de fichiers	orders_small_files	Coût de listing, planification et ouverture des fichiers	Compaction, coalesce, repartition
Partitions par date	orders_partitioned et events	Lecture efficace uniquement si la date est filtrée	Partition pruning et filtres précoces

La clé chaude 42

customer_id = 42 représente environ 22 % des commandes et user_id = 42 environ 18 % des événements. Le même identifiant est donc surreprésenté dans les deux grandes tables. Une jointure entre orders et events sur cette clé peut créer un volume extrêmement important et doit être utilisée sur un échantillon ou avec une stratégie maîtrisée.

Parcours pédagogique recommandé

Étape	Manipulation	Observation attendue
1	Lire orders_partitioned sans filtre	Toutes les partitions de date sont candidates.
2	Filtrer janvier 2026	PartitionFilters et réduction du nombre de fichiers lus.
3	Ajouter status = PAID	PushedFilters dans le plan Parquet.
4	Sélectionner quatre colonnes	ReadSchema réduit et debug_payload absent.
5	Joindre countries avec broadcast	BroadcastHashJoin et absence de shuffle de la grande table.
6	Joindre orders à customers	Observer la stratégie choisie et l'effet de la clé 42.
7	Agréger par customer_id	Identifier le skew dans les durées et volumes par tâche.
8	Comparer les deux copies des commandes	Mesurer le coût des petits fichiers.
9	Mettre en cache un résultat réutilisé	Comparer première et seconde action puis libérer le cache.

Commandes de contrôle

```
orders.printSchema()
orders.count()
orders.rdd.getNumPartitions
orders.groupBy("customer_id").count().orderBy(desc("count")).show(10)
result.explain("formatted")
```

Points d'attention

- Les données sont synthétiques et ne représentent pas des clients ou transactions réels.
- Les fonctions rand utilisent des graines fixes, ce qui rend les distributions reproductibles.
- Les volumes peuvent être réduits sur une machine peu puissante au moyen des variables d'environnement.
- OVERWRITE=true remplace les données déjà présentes dans le répertoire de sortie.
- La jointure directe orders–events peut exploser le nombre de lignes à cause de la clé 42 ; commencer par filtrer ou agréger.